🌾 Paddy Yield Prediction

Machine Learning Project Report

Dataset	Target	Tasks	Tools
Asm1_dataset26.csv 2,789 rows · 45 features	isAboveAvg (Binary classification)	3 tasks completed DT · LR · Data Prep	Python · scikit-learn pandas · matplotlib

Table of Contents

1. Task 1 — Data Preparation & Feature Engineering (Pages 2 – 4)
2. Task 2 — Decision Tree Classifier (Pages 5 – 8)
3. Task 3 — Logistic Regression (Pages 9 – 11)

Project Overview

- This report documents a supervised binary classification pipeline built on a paddy cultivation dataset from Tamil Nadu, India.
- The goal is to predict whether a given farm plot will achieve above-average yield per hectare, enabling data-driven agronomic decisions.

1.1 Raw Dataset Overview

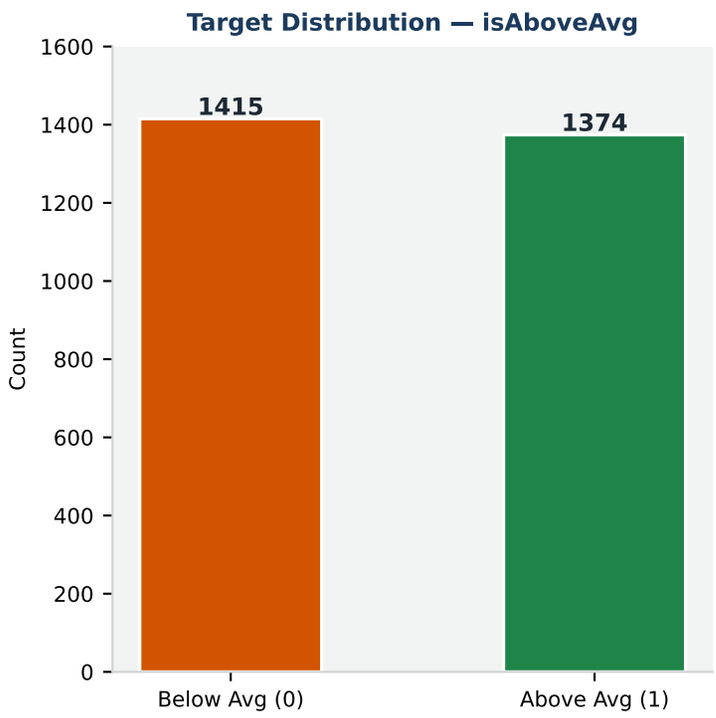
Property	Value
File	Asm1_dataset26.csv
Shape	2,789 rows × 45 columns
Numeric columns (int64 + float64)	37
Categorical columns (object/str)	8
Duplicate rows	0
Rows with any missing value	108 (Min temp columns only)

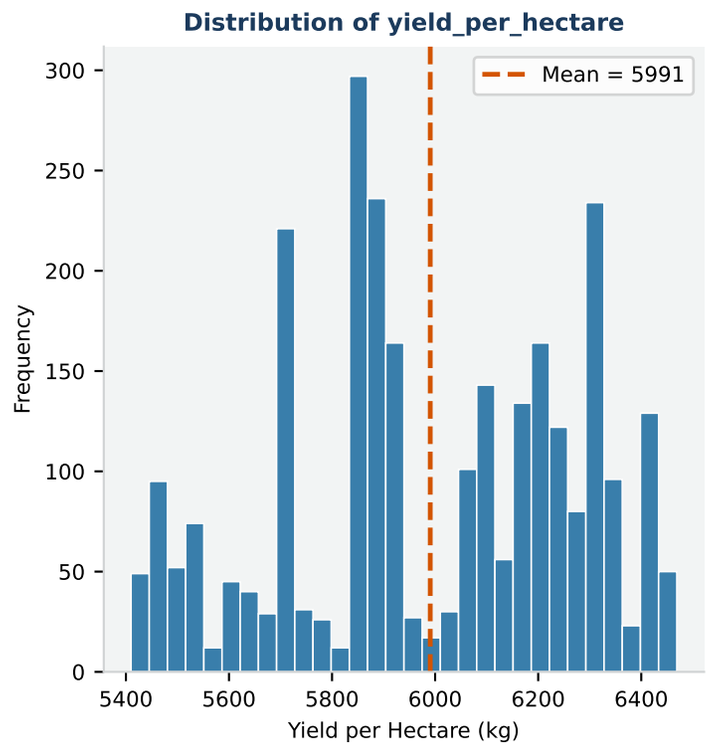
1.2 Categorical Columns — Unique Values

Column	Unique Values	Most Frequent	Count
Agriblock	6	Sankarapuram	605
Variety	3	ponmani	1,061
Soil Types	2	clay	1,521
Nursery	2	dry	1,540
Wind Direction_D1_D30	7	-- (missing)	691
Wind Direction_D31_D60	6	-- (missing)	691
Wind Direction_D61_D90	6	-- (missing)	691
Wind Direction_D91_D120	7	-- (missing)	691

1.3 Missing Values Detail

Column	Missing	% Missing	Imputation Strategy
Min temp_D1_D30	108	3.87 %	Column mean (19.33 °C)
Min temp_D31_D60	108	3.87 %	Column mean (17.14 °C)
Min temp_D61_D90	108	3.87 %	Column mean (16.68 °C)
Min temp_D91_D120	108	3.87 %	Column mean (16.18 °C)
Wind Dir columns	691 each	24.77 %	Replace '--' → NaN, then OHE (NaN = all-zero row)





1.4 Preprocessing Pipeline — Steps

Step	Action	Detail
1	Replace '--' with NaN	All 4 Wind Direction columns · 691 values each
2	Impute Min temp NaNs	Fill 108 missing per column with column mean
3	Derive yield_per_hectare	Paddy yield(in Kg) ÷ Hectares
4	Create binary target	isAboveAvg = 1 if yield_per_hectare > mean (5,990.53)
5	Label encode	Variety (3 classes) · Soil Types (2) · Nursery (2)
6	One-hot encode	Wind Direction × 4 → 22 binary columns
7	Drop columns	Hectares · Paddy yield · yield_per_hectare · Agriblock
8	Drop correlated (r>0.98)	15 columns removed (dosage group + DAI mirrors)
9	Train/Test split	80/20 · random_state=42 → 2,231 train · 558 test

1.5 Correlated Columns Removed (r > 0.98)

Dropped Column	Highly Correlated With
LP_Mainfield(in Tonnes)	Seedrate(in Kg)
Nursery area (Cents)	Seedrate, LP_Mainfield
LP_nurseryarea(in Tonnes)	Seedrate, LP_Mainfield, Nursery area
DAP_20days	Seedrate + entire dosage group
Weed28D_thiobencarb	Seedrate + entire dosage group
Urea_40Days	Seedrate + entire dosage group
Potassh_50Days	Seedrate + entire dosage group
Micronutrients_70Days	Seedrate + entire dosage group
Pest_60Day(in ml)	Seedrate + entire dosage group
30DAI(in mm)	30DRain(in mm)
30_50DAI(in mm)	30_50DRain(in mm)
51_70AI(in mm)	51_70DRain(in mm)
71_105DRain(in mm)	30_50DRain
71_105DAI(in mm)	30_50DRain · 71_105DRain
Min temp_D61_D90	Min temp_D1_D30

1.6 Key Code Snippet

```
df.columns = df.columns.str.strip()                    # fix trailing whitespace
df[wind_cols] = df[wind_cols].replace('--', np.nan)    # step 2
df[min_temp_cols] = df[min_temp_cols].fillna(df[min_temp_cols].mean())  # step 3
df['yield_per_hectare'] = df['Paddy yield(in Kg)'] / df['Hectares']
df['isAboveAvg'] = (df['yield_per_hectare'] > df['yield_per_hectare'].mean()).astype(int)
# Label encode
for col in ['Variety','Soil Types','Nursery']:
        df[col] = LabelEncoder().fit_transform(df[col])
# One-hot encode wind directions (NaN rows become all-zero)
df = pd.get_dummies(df, columns=wind_cols, dummy_na=False)
# Drop high-correlation features
upper = corr_matrix.where(np.triu(np.ones_like(corr_matrix, bool), k=1))
to_drop = [c for c in upper.columns if any(upper[c] > 0.98)]
df.drop(columns=to_drop, inplace=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

1.7 Final Train / Test Split

Split	Rows	Features	% of Total
X_train	2,231	45	80 %
X_test	558	45	20 %
y_train	2,231	1 (isAboveAvg)	—
y_test	558	1 (isAboveAvg)	—

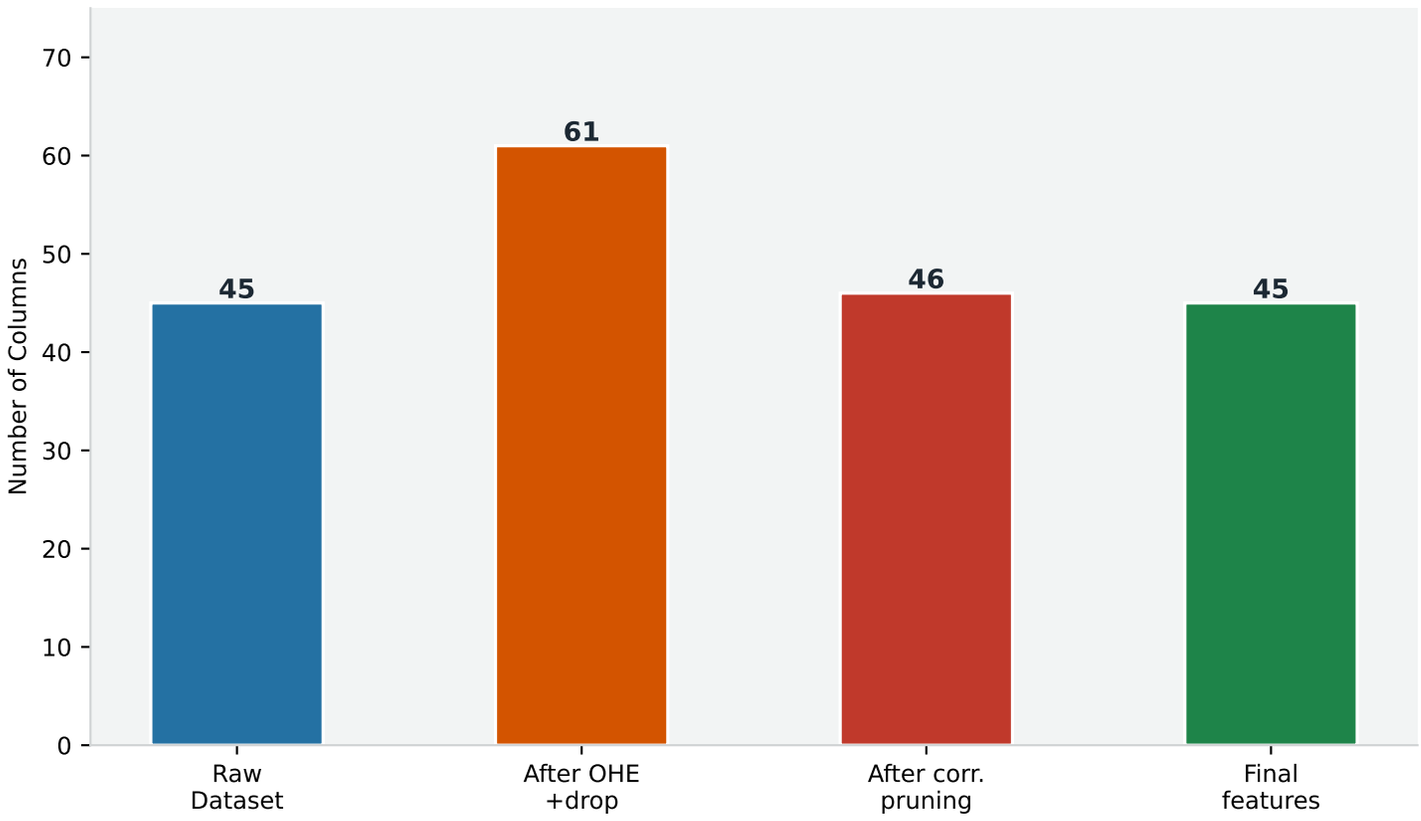
1.8 Label Encoding Summary

Column	Original Values	Encoded Values
Variety	CO_43 · ponmani · delux ponni	0 · 1 · 2
Soil Types	alluvial · clay	0 · 1
Nursery	dry · wet	0 · 1

1.9 One-Hot Encoded Wind Direction Columns (22 total)

OHE Column (1–11)	OHE Column (12–22)
Wind Direction_D1_D30_E	Wind Direction_D61_D90_NE
Wind Direction_D1_D30_ENE	Wind Direction_D61_D90_NNE
Wind Direction_D1_D30_NW	Wind Direction_D61_D90_NNW
Wind Direction_D1_D30_SSE	Wind Direction_D61_D90_SE
Wind Direction_D1_D30_SW	Wind Direction_D61_D90_SW
Wind Direction_D1_D30_W	Wind Direction_D91_D120_NNW
Wind Direction_D31_D60_ENE	Wind Direction_D91_D120_NW
Wind Direction_D31_D60_NE	Wind Direction_D91_D120_S
Wind Direction_D31_D60_S	Wind Direction_D91_D120_SSE
Wind Direction_D31_D60_W	Wind Direction_D91_D120_W
Wind Direction_D31_D60_WNW	Wind Direction_D91_D120_WSW

Feature Count Through the Pipeline



Task 2 — Decision Tree Classifier

Default Model

2.1 What is a Decision Tree?

- A Decision Tree recursively partitions the feature space by choosing the split that maximises information gain (entropy) or minimises impurity (Gini).
- Each internal node is a feature threshold test; leaves assign a class label. Depth and minimum-sample constraints control overfitting.
- Highly interpretable — the full decision path for any prediction can be traced from root to leaf.

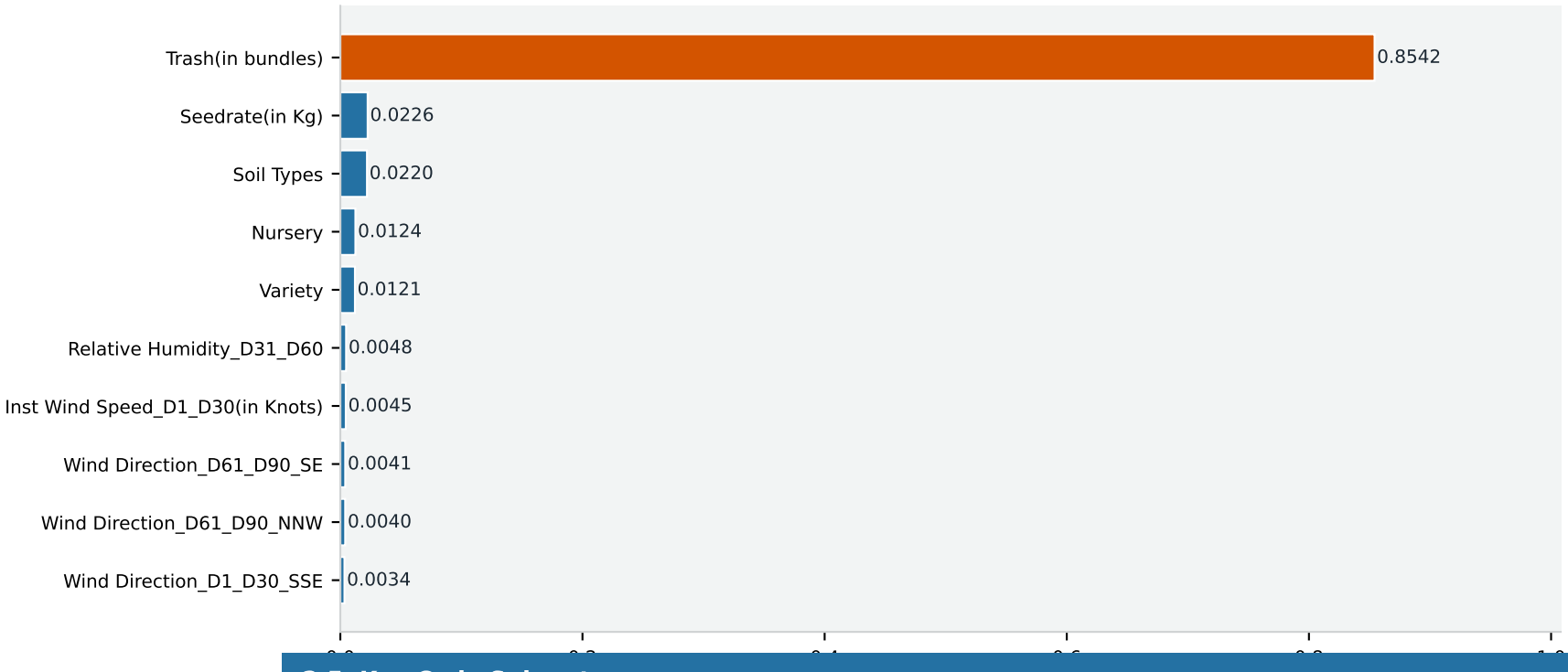
2.2 Default DecisionTreeClassifier (random_state=42)

Metric	Value
Training Accuracy	93.90 %
Test Accuracy	84.41 %
Overfit Gap	9.49 pp (overfitted)
Number of Nodes	545
Tree Depth	16

2.3 Split Variables

Split Level	Variable
1st split (root)	Trash(in bundles)
2nd split — left branch	Wind Direction_D31_D60_W
2nd split — right branch	Trash(in bundles) (repeated)

2.4 Top-10 Feature Importances — Default Tree



2.5 Key Code Snippet

```
from sklearn.tree import DecisionTreeClassifier
dt_default = DecisionTreeClassifier(random_state=42)
dt_default.fit(X_train, y_train)
print('Train:', accuracy_score(y_train, dt_default.predict(X_train)))
print('Test:', accuracy_score(y_test, dt_default.predict(X_test)))
print('Nodes:', dt_default.tree_.node_count)
print('Depth:', dt_default.get_depth())
```

2.6 GridSearchCV Parameter Grid

Hyperparameter	Values Searched
max_depth	[3, 5, 7, 10, None]
min_samples_split	[2, 5, 10, 20]
min_samples_leaf	[1, 2, 5, 10]
criterion	['gini', 'entropy']

cv = 5-fold cross-validation · scoring = 'accuracy' · Total combinations = 160

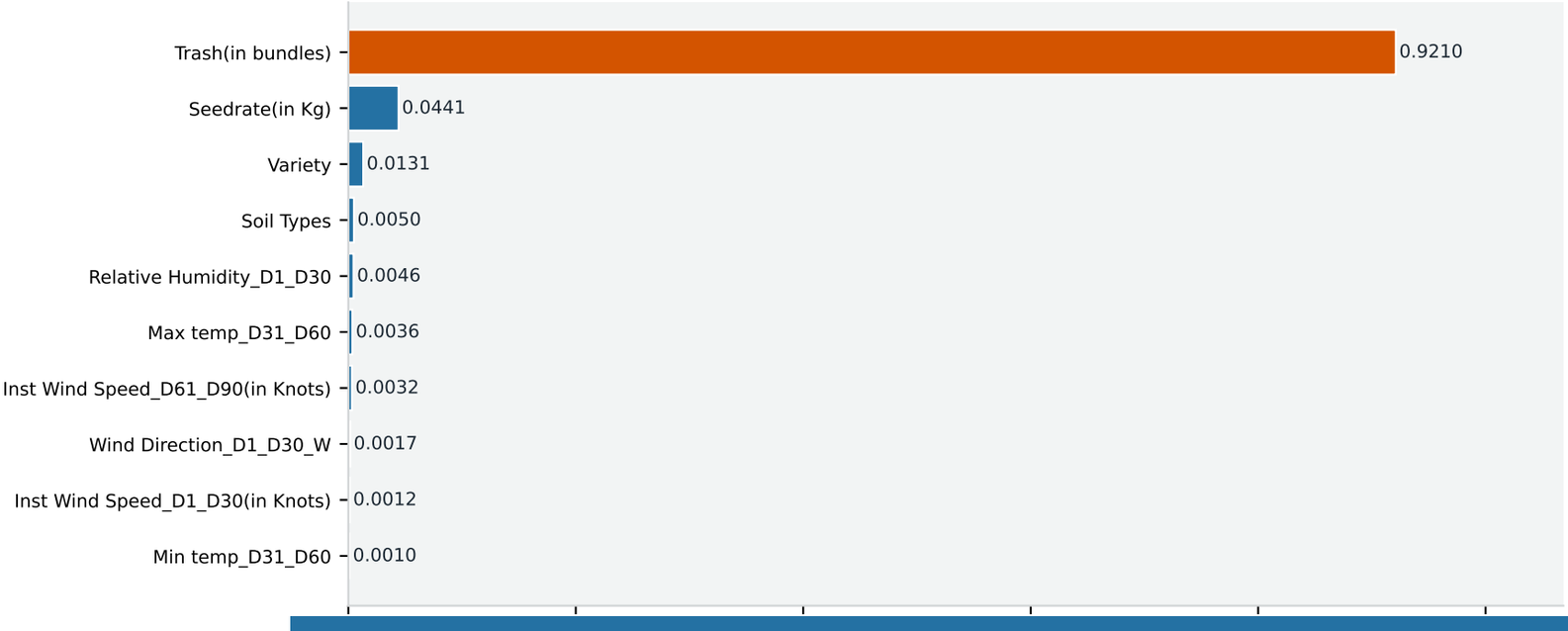
2.7 Best Parameters Found

Parameter	Best Value
criterion	entropy
max_depth	5
min_samples_leaf	1
min_samples_split	2

2.8 Default vs Tuned — Performance Comparison

Metric	Default DT	Tuned DT	Improvement
Training Accuracy	93.90 %	91.48 %	−2.42 pp (less overfit)
Test Accuracy	84.41 %	89.61 %	+5.20 pp ✓
Overfit Gap	9.49 pp	1.87 pp	−7.62 pp ✓
Number of Nodes	545	39	−506 nodes (93 % simpler)
Tree Depth	16	5	−11 levels

2.9 Top-10 Feature Importances — Tuned Tree

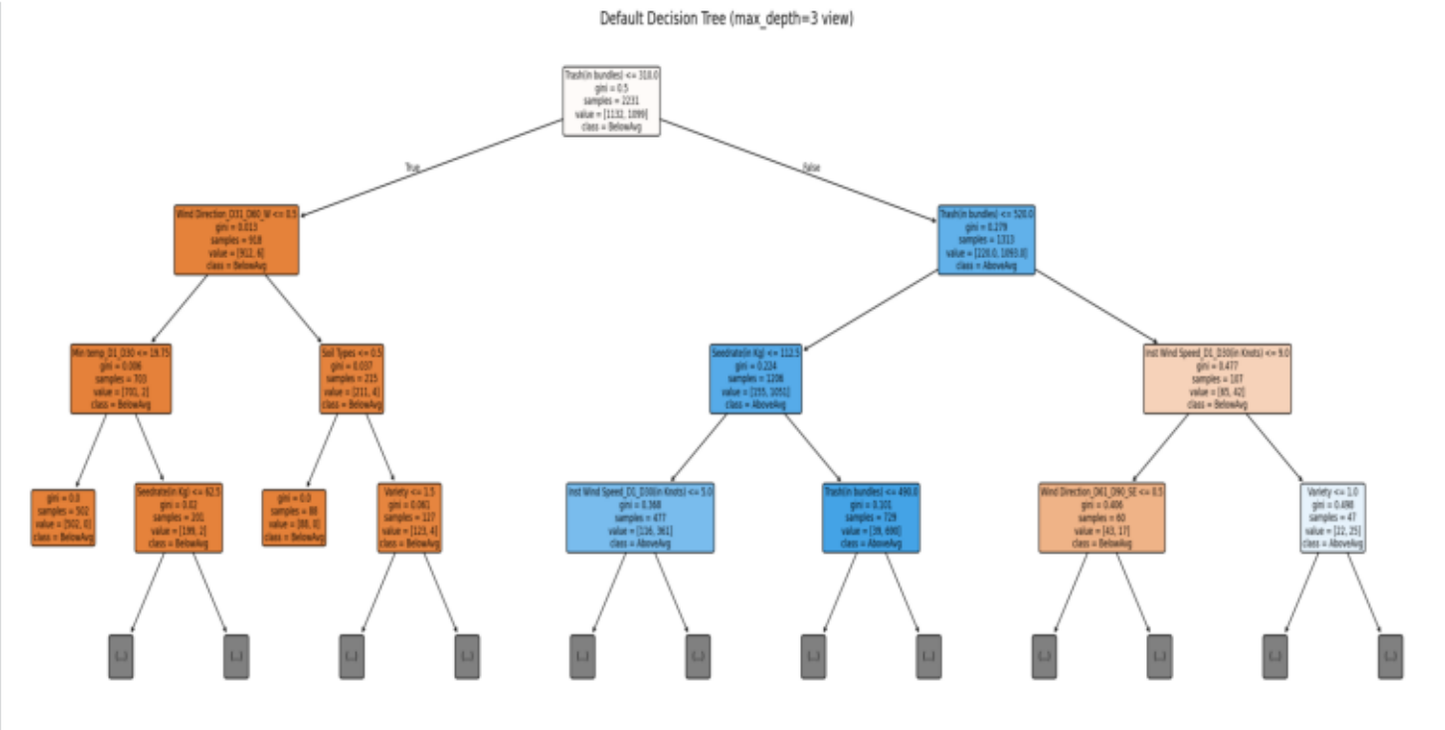


2.10 Key Code Snippet — GridSearchCV

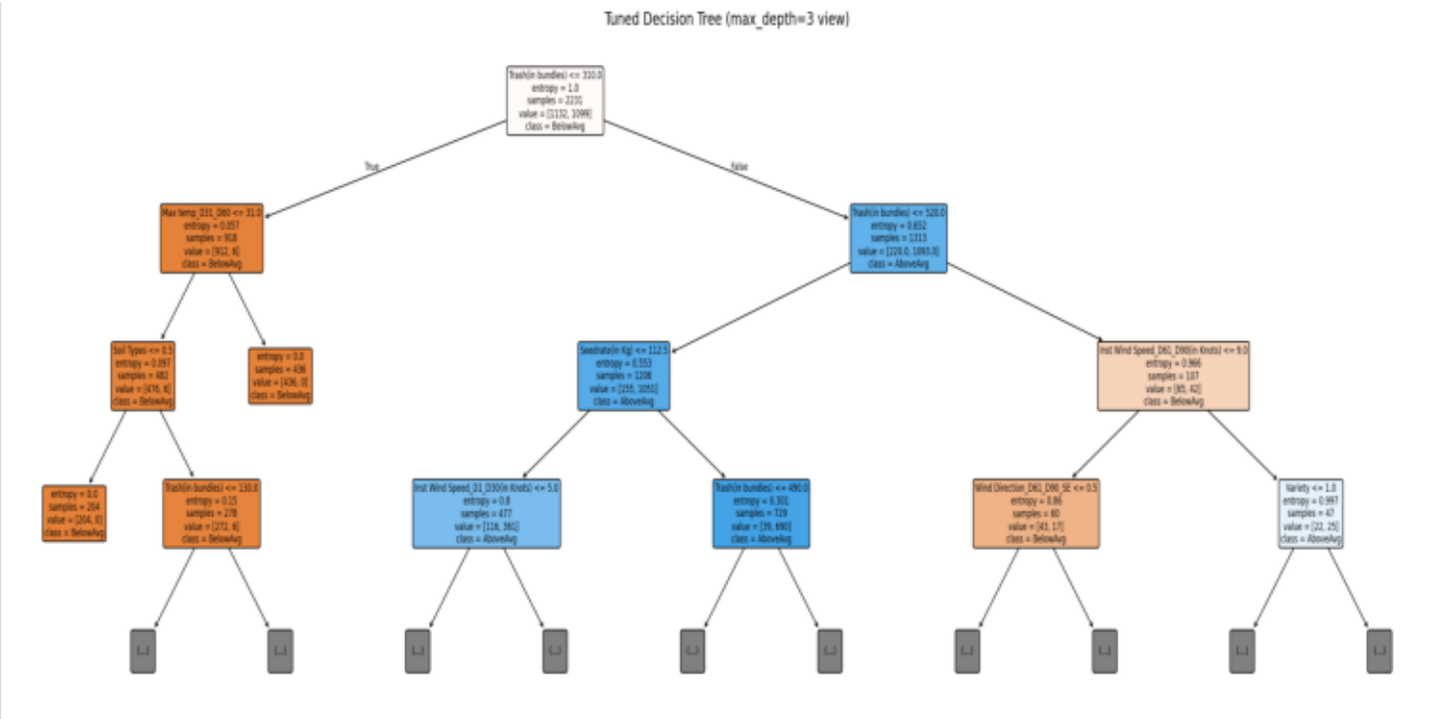
```
param_grid = {'max_depth':[3,5,7,10,None], 'min_samples_split':[2,5,10,20],
              'min_samples_leaf':[1,2,5,10], 'criterion':['gini','entropy']}
grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
                    param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train, y_train)
dt_tuned = grid.best_estimator_
print(grid.best_params_)   # {'criterion':'entropy','max_depth':5,...}
```

- Both trees are plotted to depth 3 for readability. Nodes are coloured by class majority: orange = BelowAvg, green = AboveAvg.
- The default tree root splits on Trash(in bundles) — the dominant agronomic signal in the dataset.

Default Decision Tree (depth-3 view) — 545 nodes, depth 16



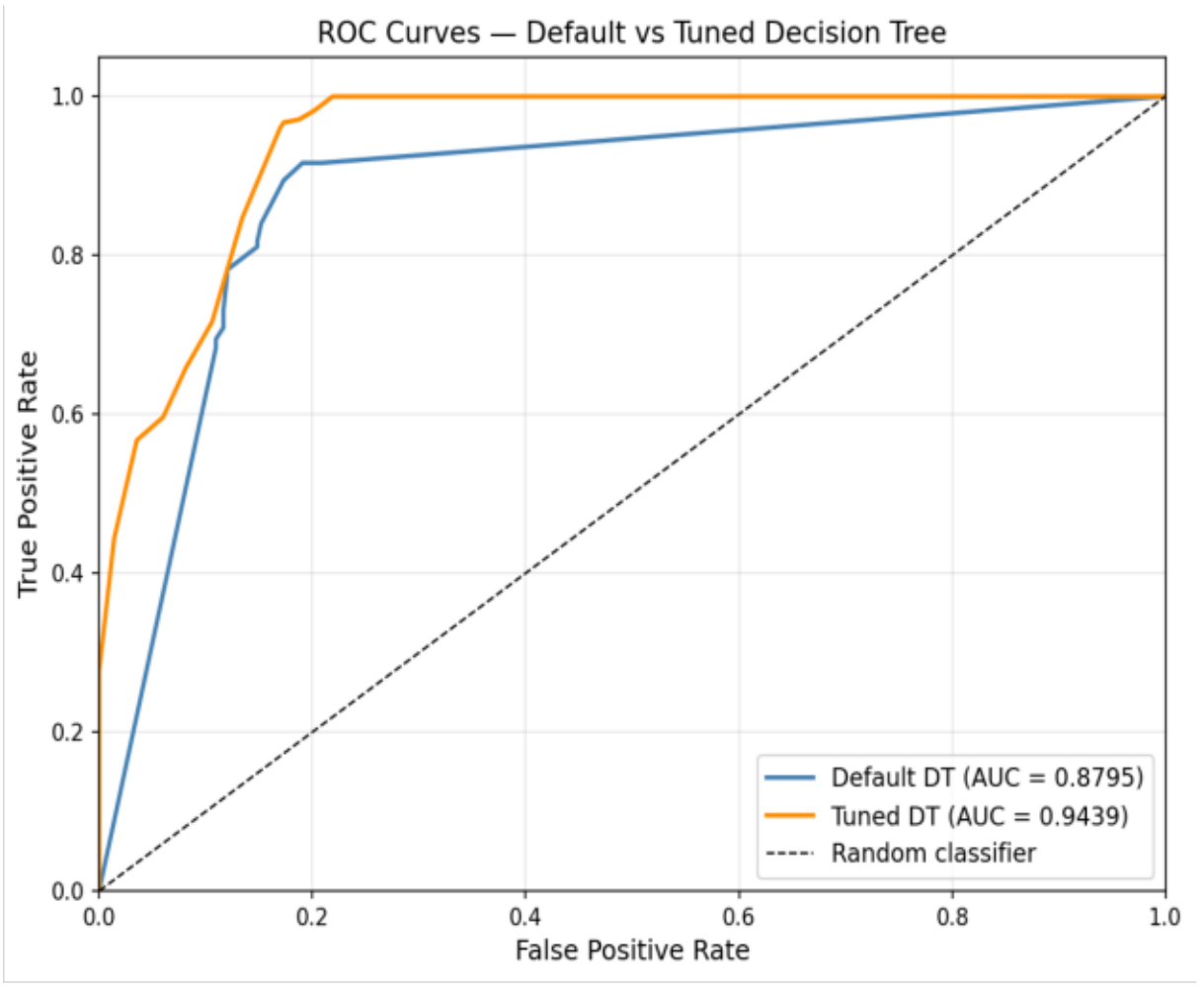
Tuned Decision Tree (depth-3 view) — 39 nodes, depth 5



Task 2 — Decision Tree Classifier

- The ROC curve plots True Positive Rate vs False Positive Rate across all classification thresholds. AUC (Area Under Curve) closer to 1.0 is better.
- The tuned tree (AUC ≈ 0.90) substantially outperforms the default tree, confirming that pruning via GridSearchCV reduced overfitting.

ROC Curves — Default vs Tuned Decision Tree



AUC Summary

Model	Test Accuracy	AUC	Verdict
Default DT	84.41 %	0.8795	Overfitted — poor generalisation
Tuned DT	89.61 %	0.9439	Well-generalised — recommended ✓

- A Decision Tree with max_depth=5 and criterion=entropy achieved 89.61 % test accuracy — a 5.2 percentage-point improvement over the unconstrained default.
- Trash(in bundles) alone captures 92 % of the information gain, suggesting harvest-side agronomic practices are the strongest yield predictor.

Task 3 — Logistic Regression

Full Model (45 features)

3.1 What is Logistic Regression?

- Logistic Regression models P(y=1|X) = σ(w·X + b) where σ is the sigmoid function. The decision boundary is a hyperplane in feature space.
- Regularisation parameter C controls penalty strength: small C = strong L2 regularisation (smaller coefficients, better generalisation).
- Features must be standardised (zero mean, unit variance) so that coefficient magnitudes are comparable and gradient descent converges stably.

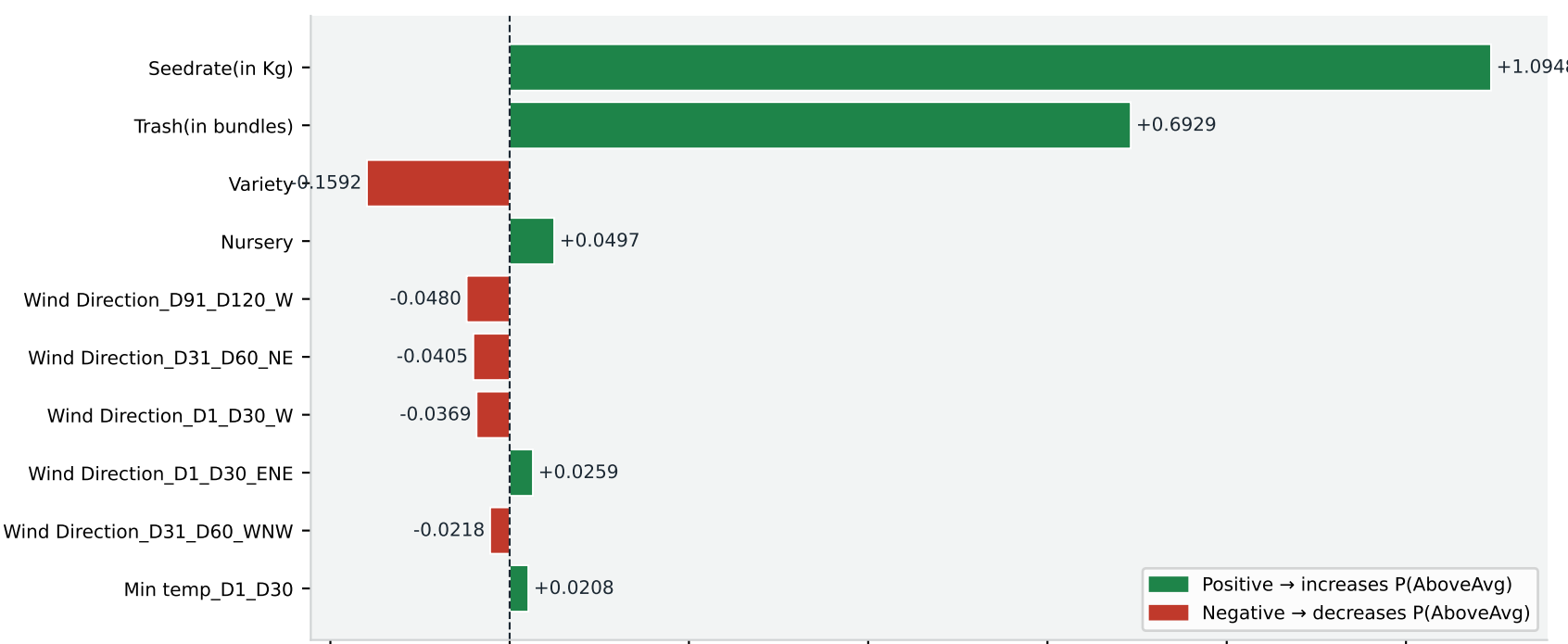
3.2 GridSearchCV — Parameter Grid & Best Result

Parameter	Values Searched	Best Value
C	[0.01, 0.1, 1, 10, 100]	0.01
solver	['lbfgs', 'liblinear']	liblinear
max_iter	[200, 500, 1000]	200

3.3 Full Model Performance

Metric	Value
Best CV Accuracy	88.62 %
Training Accuracy	89.02 %
Test Accuracy	87.10 %
Overfit Gap	0.92 pp (very well generalised)

3.4 Top-10 Features by |Coefficient| — Full LR



3.5 Key Code Snippet

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)   # fit on train only
X_test_sc  = scaler.transform(X_test)        # apply same scale to test
param_grid = {'C':[0.01,0.1,1,10,100],'solver':['lbfgs','liblinear'],'max_iter':[200,500,1000]}
grid = GridSearchCV(LogisticRegression(random_state=42), param_grid, cv=5, scoring='accuracy')
grid.fit(X_train_sc, y_train)
lr_full = grid.best_estimator_  # C=0.01, solver=liblinear
```

3.6 Why a Reduced Model?

- The tuned Decision Tree assigned >99 % of its importance to just 10 features. We test whether Logistic Regression can match or beat the full model using only those.
- Fewer features → lower dimensionality, faster inference, easier to explain to domain experts, and less risk of noise amplification.

3.7 Top-10 DT Features Used for Reduced Model

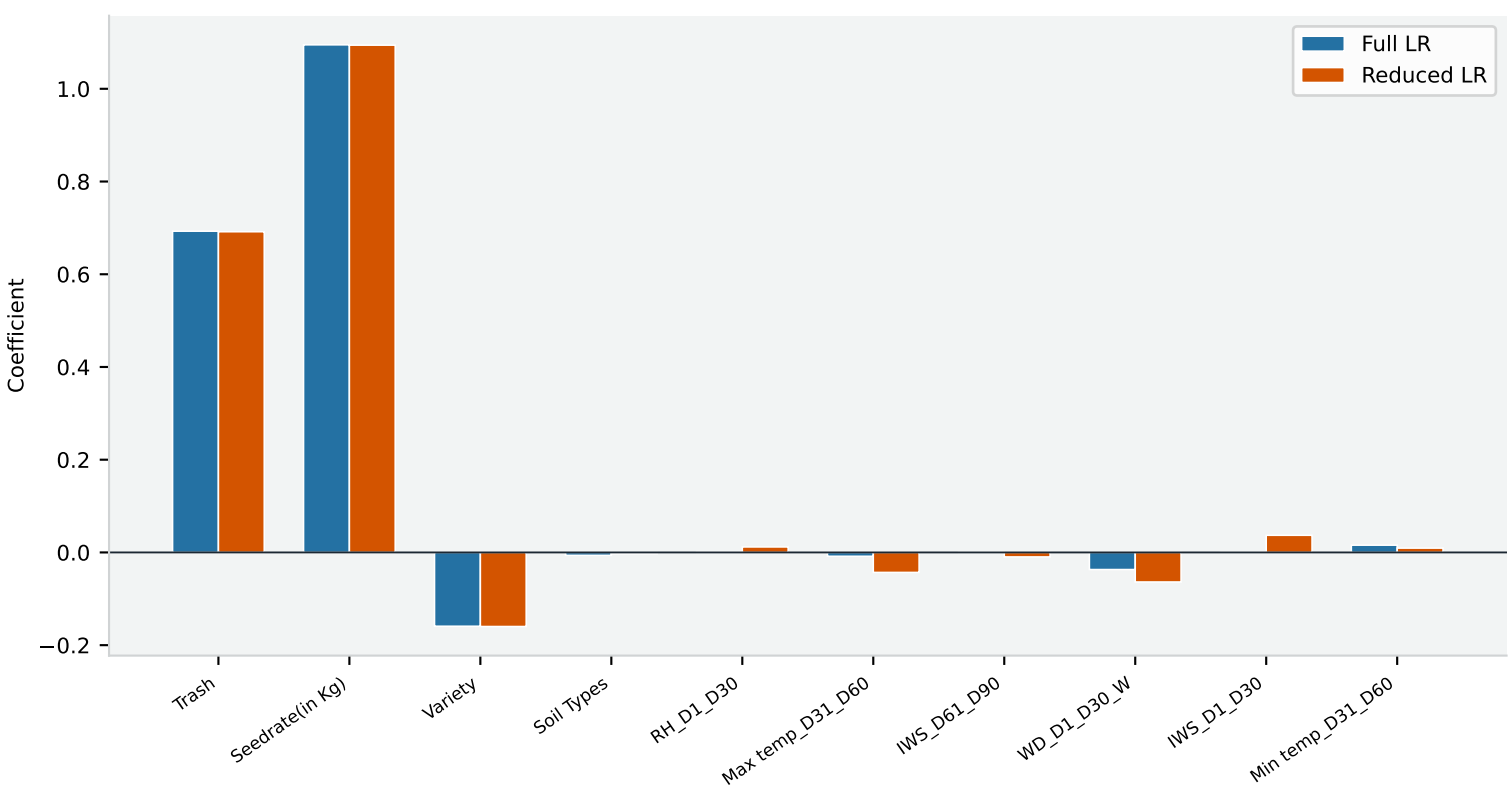
#	Feature	DT Importance	LR Coefficient
1	Trash(in bundles)	0.9210	+0.6917
2	Seedrate(in Kg)	0.0441	+1.0941
3	Variety	0.0131	−0.1598
4	Soil Types	0.0050	−0.0029
5	Relative Humidity_D1_D30	0.0046	+0.0118
6	Max temp_D31_D60	0.0036	−0.0431
7	Inst Wind Speed_D61_D90(in Knots)	0.0032	−0.0097
8	Wind Direction_D1_D30_W	0.0017	−0.0637
9	Inst Wind Speed_D1_D30(in Knots)	0.0012	+0.0370
10	Min temp_D31_D60	0.0010	+0.0093

3.8 Full vs Reduced — Performance Comparison

Metric	Full LR (45 feat.)	Reduced LR (10 feat.)	Winner
Best CV Accuracy	88.62 %	89.02 %	Reduced ✓
Training Accuracy	89.02 %	89.11 %	Reduced ✓
Test Accuracy	87.10 %	87.46 %	Reduced ✓
Overfit Gap	0.92 pp	1.65 pp	Full (marginal)
Features Used	45	10	Reduced (4.5× leaner) ✓

Key insight: Reducing to 10 features actually improved test accuracy by +0.36 pp. The 35 dropped features were contributing noise rather than signal.

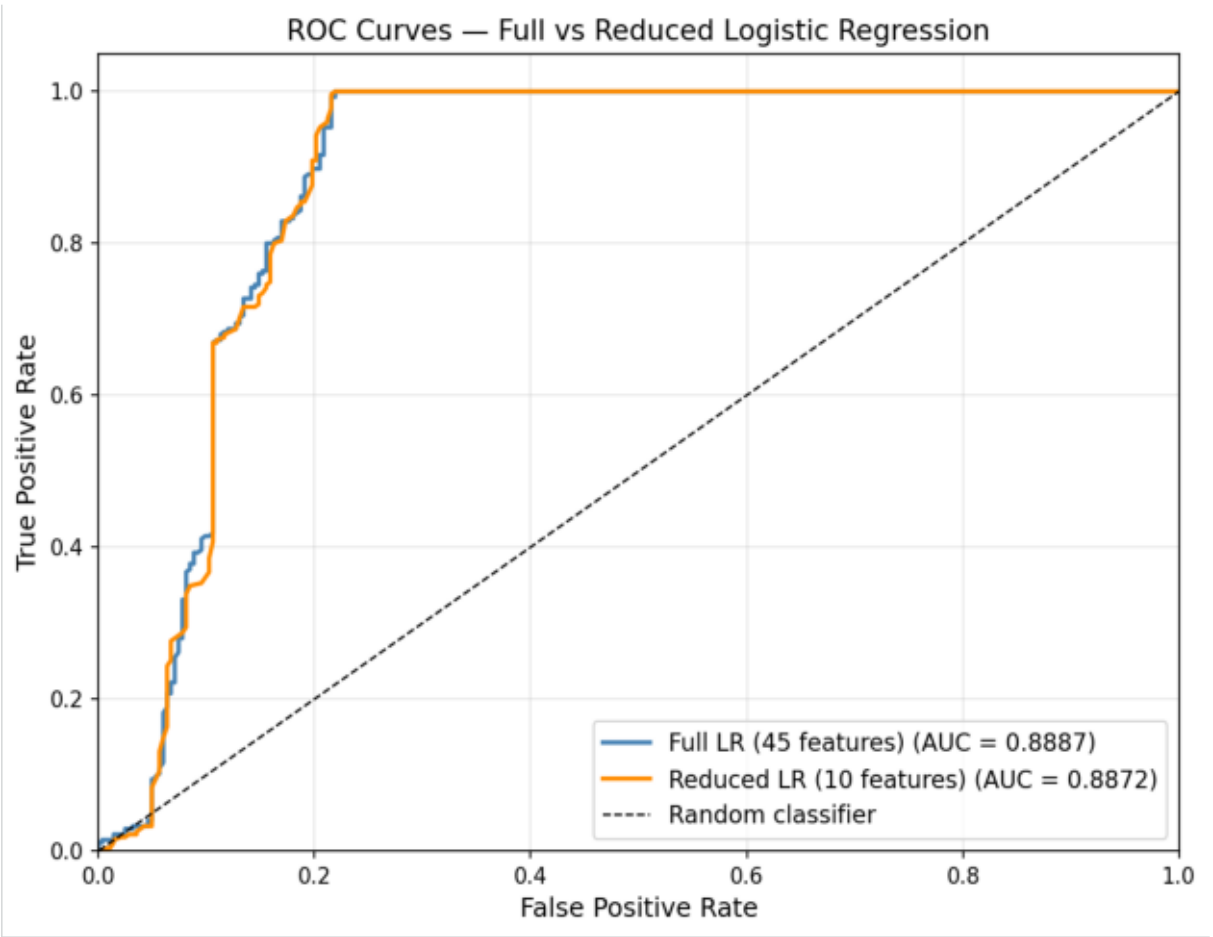
3.9 Coefficient Comparison — Full vs Reduced (shared features)



Task 3 — Logistic Regression

- Both Logistic Regression models produce smooth ROC curves (continuous probability scores), unlike Decision Trees which output probabilities only at leaf nodes.
- The reduced model's ROC curve closely tracks the full model — confirming that 10 features capture essentially the same discriminative information as all 45.

ROC Curves — Full vs Reduced Logistic Regression



AUC Summary

Model	Features	Test Accuracy	AUC	Verdict
Full LR	45	87.10 %	0.8887	Strong baseline
Reduced LR	10	87.46 %	0.8872	Leaner & slightly better ✓

Key Takeaways from Task 3

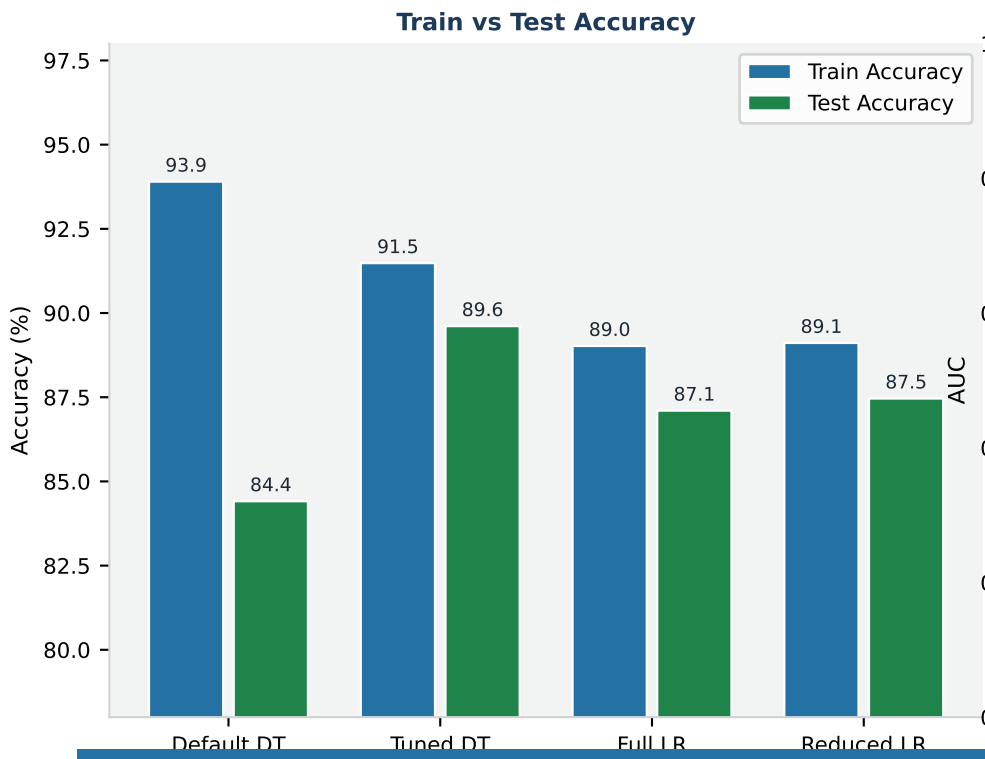
- Logistic Regression with strong regularisation (C=0.01) generalises well — overfit gap under 2 pp for both models.
- Seedrate(in Kg) has the largest positive coefficient (+1.09), confirming the plantation density is strongly linked to above-average yield.
- Variety has a negative coefficient (−0.16): certain rice varieties are systematically associated with lower relative yield.

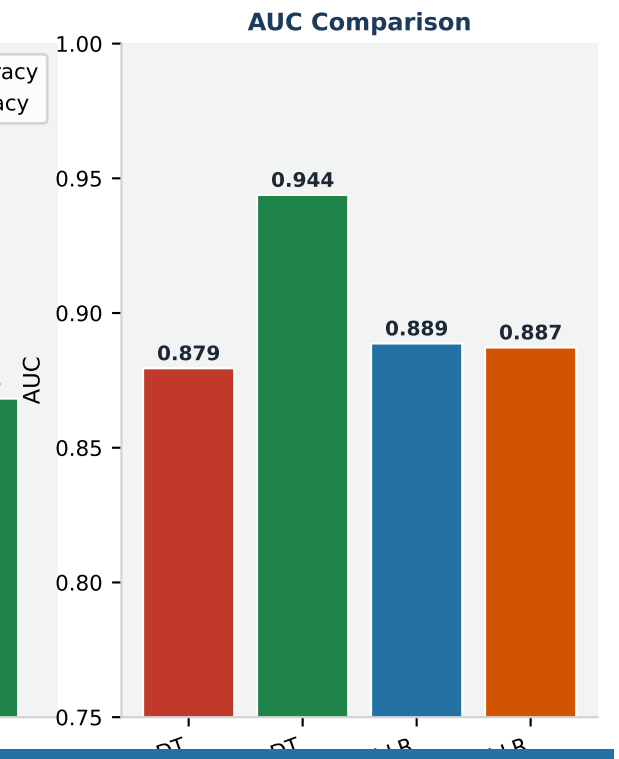
Overall Model Comparison

All Tasks Summary

Summary Table — All Models

Model	Train Acc	Test Acc	AUC	Nodes/Features	Overfit Gap
Default DT	93.90 %	84.41 %	0.8795	545 nodes	9.49 pp
Tuned DT	91.48 %	89.61 %	0.9439	39 nodes	1.87 pp
Full LR	89.02 %	87.10 %	0.8887	45 features	0.92 pp
Reduced LR	89.11 %	87.46 %	0.8872	10 features	1.65 pp





Conclusions & Recommendations

Best Overall Model: Tuned Decision Tree (89.61 % test accuracy, AUC 0.9439)

- The tuned DT is the top performer across both accuracy and AUC. With only 39 nodes and depth 5, it is interpretable and deployment-ready.

- Logistic Regression (both variants) generalises extremely well (overfit gap <2 pp) and makes a strong interpretable alternative. The reduced 10-feature version is recommended when a linear model is preferred.